

Padrões de Projeto (Template Method)

QXD0068 - Reuso de Software

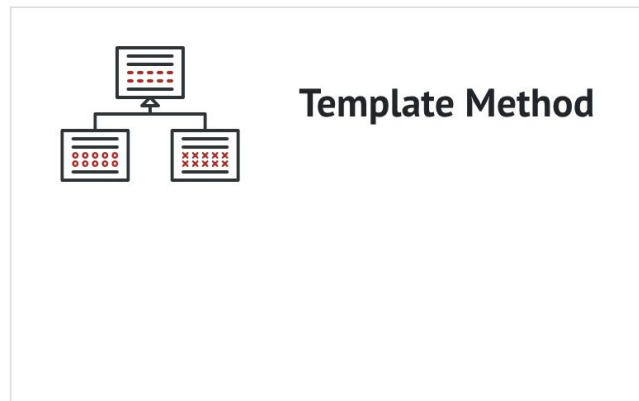
Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução
- Padrão Template Method
- Problema
- Solução
- Analogia do Mundo Real
- Estrutura do Padrão
- Exemplo em Kotlin e Python
- Aplicabilidade
- Prós e Contras
- Implementação

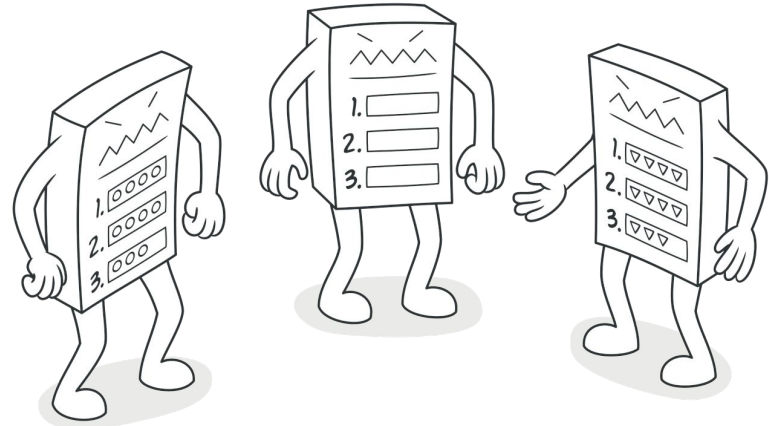


Introdução

- O Template Method (ou Método Padrão) é um padrão de projeto comportamental que tem como objetivo definir o esqueleto de um algoritmo em uma superclasse, permitindo que subclasses personalizem partes específicas desse algoritmo sem alterar sua estrutura geral.
- Esse padrão é muito útil quando várias classes compartilham a mesma sequência lógica de operações, mas diferem na implementação de alguns passos. Dessa forma, o Template Method promove reuso de código, extensibilidade e redução de duplicação.

Padrão Template Method

- A ideia central do Template Method é fornecer um método padrão (template) que define a ordem das etapas de um processo. Cada uma dessas etapas é representada por métodos — alguns com implementação padrão, outros abstratos.
- As subclasses podem então sobrescrever os métodos abstratos (ou opcionais) para alterar o comportamento de etapas específicas, sem modificar o método principal que representa o fluxo completo.
- Assim, o método template atua como um roteiro fixo, e as subclasses apenas preenchem os detalhes específicos.



Problema

- A primeira versão da aplicação podia funcionar somente com arquivos DOC. Na versão seguinte, ela era capaz de suportar arquivos CSV. Um mês depois, você a “ensinou” a extrair dados de arquivos PDF.
- Em algum momento você percebeu que todas as três classes tem muito código parecido. Embora o código para lidar com vários formatos seja inteiramente diferente em todas as classes, o código para processamento de dados e análise é quase idêntico. Não seria bacana se livrar da duplicação de código, deixando a estrutura do algoritmo intacta?
- Havia outro problema relacionado com o código cliente que usou essas classes. Ele tinha muitas condicionais que pegavam um curso de ação apropriado dependendo da classe do objeto processador. Se todas as três classes processantes tiverem uma interface comum ou uma classe base, você poderia eliminar as condicionais no código cliente e usar polimorfismo quando chamar métodos em um objeto sendo processado.

-
- ```

classDiagram
 class DocDataMiner {
 +mine(path)
 }
 class CSVDataMiner {
 +mine(path)
 }
 class PDFDataMiner {
 +mine(path)
 }
 class Doc
 class CSV
 class PDF
 class DATA

 Doc --> DocDataMiner
 CSV --> CSVDataMiner
 PDF --> PDFDataMiner
 DocDataMiner --> DATA
 CSVDataMiner --> DATA
 PDFDataMiner --> DATA

```
- The diagram illustrates the Factory Method pattern for data processing. It shows three document types (Doc, CSV, PDF) and their corresponding miners (DocDataMiner, CSVDataMiner, PDFDataMiner). Each miner has a 'mine(path)' method. The process involves opening a file, extracting data, parsing it, analyzing it, sending a report, and closing the file. The results are stored in a 'DATA' database.
- DocDataMiner**
- ```

file = openFile(path)
rawData = extractDocData(file)
data = parseDocData(rawData)
analysis = analyzeData(data)
sendReport(analysis)
closeFile(file)
  
```
- CSVDataMiner**
- ```

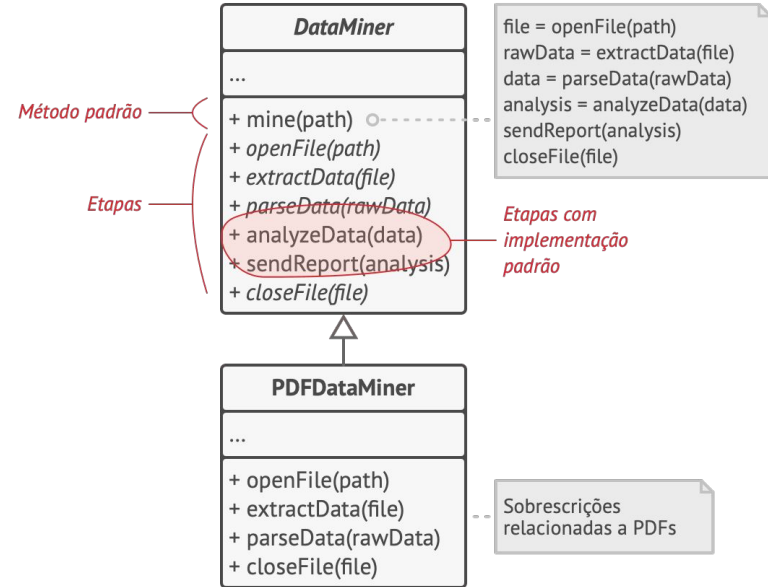
file = openFile(path)
rawData = extractCSVData(file)
data = parseCSVData(rawData)
analysis = analyzeData(data)
sendReport(analysis)
closeFile(file)

```
- PDFDataMiner**
- ```

file = openFile(path)
rawData = extractPDFData(file)
data = parsePDFData(rawData)
analysis = analyzeData(data)
sendReport(analysis)
closeFile(file)
  
```
- DATA**

Solução

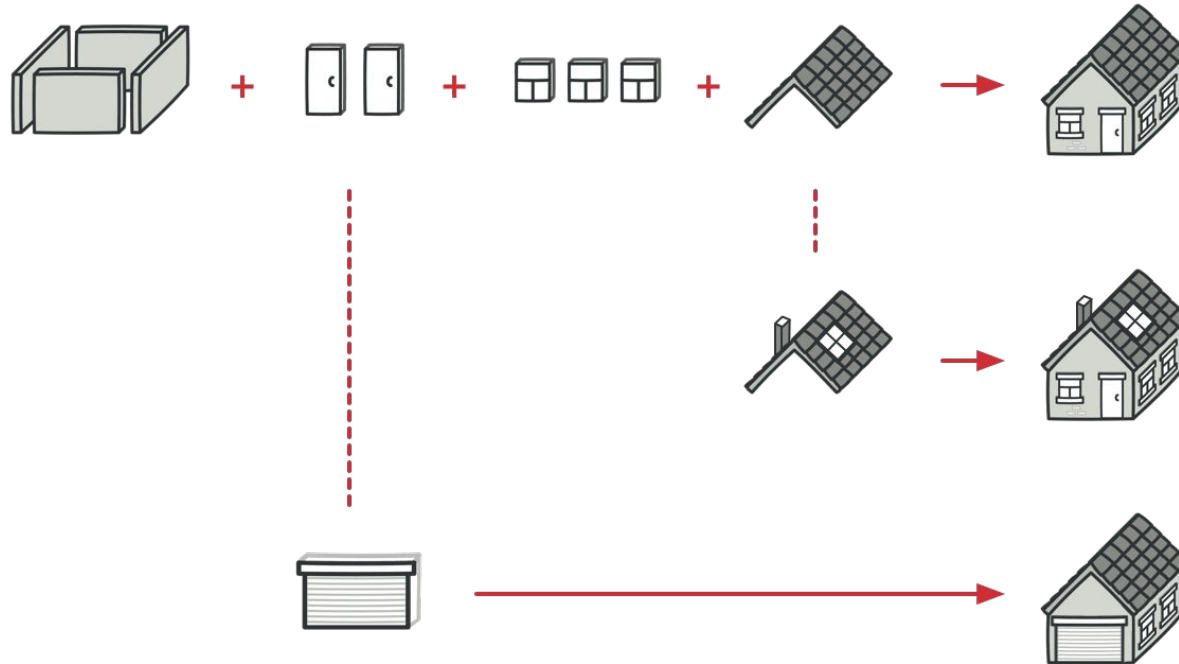
- **O Template Method propõe:**
 - Dividir o algoritmo em etapas menores.
 - Definir um método “template” na superclasse que chama essas etapas na ordem correta.
 - Permitir que as subclasses implementem ou sobrescrevam apenas as etapas necessárias.
- **Existem:**
 - Etapas abstratas: obrigatoriamente implementadas pelas subclasses.
 - Etapas opcionais: com implementação padrão.
 - Hooks (ganchos): métodos vazios que podem ser sobrescritos para alterar o comportamento em pontos específicos.



Analogia do Mundo Real

- Um exemplo prático é o modelo de construção de casas em massa.
- A planta arquitetônica define uma sequência padrão de etapas — fundações, estrutura, paredes, encanamento, parte elétrica, acabamento — mas permite ajustes personalizados em pontos específicos, como o tipo de piso, o layout interno ou a cor das paredes.
- Da mesma forma, o Template Method define a sequência fixa (o plano da casa), enquanto as subclasses representam as variações de acordo com o gosto do cliente, mantendo a base inalterada.

Analogia do Mundo Real



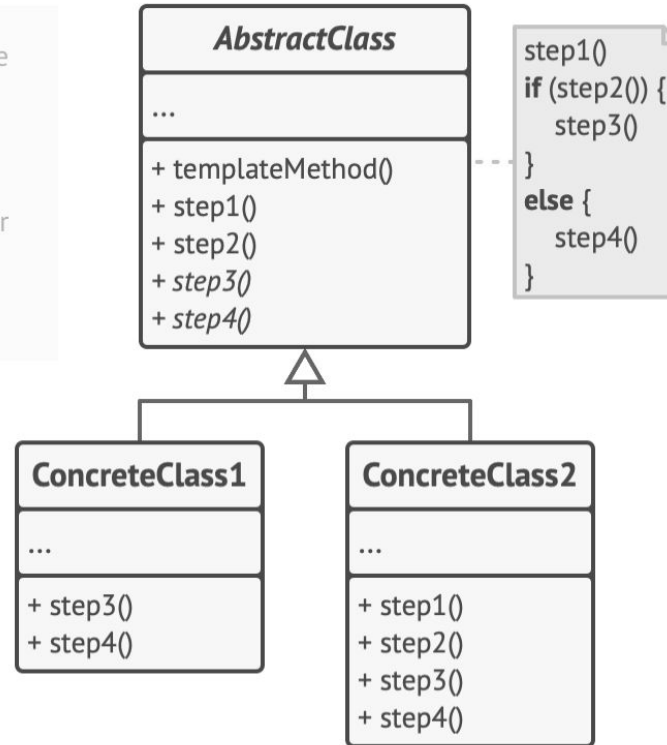
Estrutura do Padrão

- **Classe Abstrata (Template):**
 - Declara o método template, que define a sequência de execução do algoritmo.
 - Contém métodos abstratos que devem ser implementados pelas subclasses.
 - Pode conter métodos concretos com implementações padrão.
 - Pode incluir hooks (ganchos) — métodos opcionais que as subclasses podem sobrescrever.

- **Classe Concreta:**
 - Implementa os métodos abstratos.
 - Pode sobrescrever hooks ou métodos opcionais.
 - Não altera o método template.

1 A **Classe Abstrata** declara métodos que agem como etapas de um algoritmo, bem como o próprio método padrão que chama esses métodos em uma ordem específica. Os passos podem ser declarados como `abstratos` ou ter alguma implementação padrão.

2 As **Classes Concretas** podem sobrescrever todas as etapas, mas não o próprio método padrão.



```
// Classe abstrata que define o algoritmo
abstract class DataMiner {
    fun mine(path: String) {
        openFile(path)
        extractData()
        parseData()
        closeFile()
        analyzeData()
        sendReport()
    }

    abstract fun openFile(path: String)
    abstract fun extractData()
    abstract fun parseData()
    abstract fun closeFile()

    // Método opcional (hook)
    open fun analyzeData() {
        println("Analisando dados padrão...")
    }

    // Método concreto
    fun sendReport() {
        println("Relatório enviado com sucesso.")
    }
}

// Subclasse concreta para CSV
class CsvDataMiner : DataMiner() {
    override fun openFile(path: String) = println("Abrindo arquivo CSV: $path")
    override fun extractData() = println("Extraindo dados do CSV...")
    override fun parseData() = println("Interpretando dados CSV...")
    override fun closeFile() = println("Fechando arquivo CSV...")
    override fun analyzeData() = println("Análise estatística dos dados CSV concluída.")
}
```

Exemplo em Kotlin

```
from abc import ABC, abstractmethod

class DataMiner(ABC):
    def mine(self, path):
        self.open_file(path)
        self.extract_data()
        self.parse_data()
        self.close_file()
        self.analyze_data()
        self.send_report()

    @abstractmethod
    def open_file(self, path): pass

    @abstractmethod
    def extract_data(self): pass

    @abstractmethod
    def parse_data(self): pass

    @abstractmethod
    def close_file(self): pass

    def analyze_data(self):
        print("Analisando dados com método padrão...")

    def send_report(self):
        print("Relatório enviado com sucesso.")

class PdfDataMiner(DataMiner):
    def open_file(self, path): print(f"Abrindo PDF: {path}")
    def extract_data(self): print("Extraindo dados do PDF...")
    def parse_data(self): print("Convertendo conteúdo PDF em texto estruturado...")
    def close_file(self): print("Fechando PDF...")
    def analyze_data(self): print("Análise semântica de documentos PDF concluída.")
```

Exemplo em Python

Aplicabilidade

- **Use o Template Method quando:**
 - Você quer permitir que subclasses redefinam partes específicas de um algoritmo sem alterar sua estrutura global.
 - Há múltiplas classes com algoritmos semelhantes, mas com pequenas variações.
 - Deseja eliminar código duplicado, reunindo comportamento comum em uma superclasse base.
 - Precisa garantir uniformidade na execução de um processo, com flexibilidade controlada nas etapas.

Prós e Contras

- **Vantagens**

- Promove reuso de código ao concentrar a lógica comum em uma superclasse.
- Facilita a extensão do comportamento sem modificar a estrutura do algoritmo.
- Reduz duplicação de código e melhora a organização e manutenção.
- Permite ganchos (hooks) para adicionar comportamentos opcionais.

- **Desvantagens**

- Aumenta a dependência entre classes devido ao uso intensivo de herança.
- Pode violar o Princípio de Substituição de Liskov se as subclasses alterarem etapas de forma inadequada.
- Torna-se difícil de manter se o algoritmo tiver muitas etapas ou subclasses complexas.
- Menos flexível que padrões baseados em composição, como o Strategy.



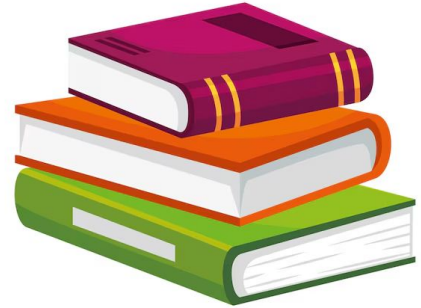
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

